

Course Outline: Data Flow in React and Next.js

Title: Data Flow in React and Next.js **Learnpack:** + ¿Cómo es el flujo de datos en una aplicación de React y Next.js? **Prerequisite:** Skill 12 — Interactive Interfaces with React + Next.js (Week 5, Day 15) **Target Audience:** Beginner developers who have built React components and understand hooks, and are now ready to think about how data moves and persists across their application **Total Lessons:** 17 **Estimated Total Length:** ~8,500 words **Format:** Mixed (35% hands-on = 6 lessons)

Course Description

You've built components, wired up hooks, and made things appear on the screen. But as your apps grow, a new question emerges: where does data actually live, and how does it move from place to place? This course answers that question with clarity and practice.

We'll start by looking at MVC — a mental model that helps you separate *what your app knows* (the store) from *what your app does with that knowledge* (actions). You'll learn why having a single source of truth makes your app easier to reason about, debug, and scale. Then we'll explore browser storage: the difference between LocalStorage and SessionStorage, when it makes sense to use either, and the anti-pattern that trips up almost every beginner.

Finally, you'll learn how to control data dependencies using React's `use` hook and understand one-way data flow — the core principle that makes React predictable. By the end, you'll be able to design data flow intentionally, not by accident.

Course Philosophy

- This course emphasizes:
- **Mental models before code:** Understand *why* before *how* — knowing the shape of a problem makes the solution obvious
 - **Path of least resistance:** Start with the simplest approach that works, then add structure only when the need is clear
 - **Intentional design:** Data flow should be a deliberate architectural decision, not an afterthought discovered during debugging
-

Course Statistics Summary

Section	Lessons	Estimated Words
00 — Welcome	1	~450
01 — MVC in React	4	~2,000
02 — Browser Storage	4	~2,000
03 — Data Dependencies	4	~2,000

Section	Lessons	Estimated Words
04 — Assessment	2	~1,600
05 — What's Next	1	~450
Total	17	~8,500

Section 00: Welcome

00.0 How Data Moves in Your App 📖 (~450 words)

Learning Objectives:

- Recognize that data flow is a design decision, not an accident
- Identify the three core topics this course will cover: MVC, browser storage, and data dependencies
- Connect this course to what was learned in the previous learnpack about hooks and components

Content Outline:

- The problem that appears when apps grow: where does data live?
- A brief map of what this course covers and why each topic matters
- How this connects to hooks and components already learned

Transitions: This lesson sets up the mindset for everything that follows. Students end knowing what questions this course will answer, which primes them to receive the answers meaningfully.

Key Principles:

- Data flow should be designed intentionally — not discovered through debugging
- The bigger an app gets, the more important it is to know exactly where each piece of data lives

Examples:

- A shopping cart that loses its items when you navigate to another page — because nobody thought about where cart data should live
 - A user's language preference that resets every time they refresh — because nobody chose where to persist it
-

Section 01: MVC in React

01.0 What Is MVC and Why It Matters 📖 (~500 words)

Learning Objectives:

- Define MVC (Model-View-Controller) and explain each of its three parts
- Explain why separating concerns makes apps easier to maintain
- Recognize how MVC maps to a typical React application

Content Outline:

- What MVC stands for and where the pattern comes from
- Model: the data and business logic
- View: what the user sees (React components)
- Controller: the bridge between data and view (actions)
- Why this separation matters even in small apps

Transitions: This lesson establishes the MVC vocabulary. The next lesson zooms in on the Model layer — specifically the Store.

Key Principles:

- Separation of concerns: each part of your app should have one clear job
- MVC is a mental model, not a strict rule — React adapts it in its own way

Examples:

- A todo list: the array of todos is the Model, the `<TodoList />` component is the View, and the `addTodo()` function is the Controller
 - A weather app: fetched weather data is the Model, the UI showing temperature is the View, and the function that triggers the fetch is the Controller
-

01.1 The Store: Single Source of Truth 📖 (~500 words)

Learning Objectives:

- Define what a Store is in the context of a React app
- Explain why having a single source of truth prevents data inconsistencies
- Identify what kinds of data belong in a Store versus local component state

Content Outline:

- What a Store is: a centralized place where your app's data lives
- Why "single source of truth" matters: when data exists in multiple places, it gets out of sync
- The difference between global state (in the Store) and local state (inside a component)
- Examples of data that belongs in the Store versus data that stays local

Transitions: Now that students understand what the Store holds, the next lesson covers how to change what's in it — through Actions.

Key Principles:

- A Store is only modified by controlled, predictable functions — never directly
- Local state is for UI concerns (is this dropdown open?); the Store is for app-wide data (who is the current user?)

Examples:

- Global Store data: the logged-in user's name, the contents of a shopping cart, a list of fetched products
 - Local component state: whether a modal is open, which tab is currently selected, the value of a text input before submission
-

01.2 Actions: Talking to the Store 📖 (~500 words)

Learning Objectives:

- Define what an Action is and how it modifies the Store
- Explain why the Store should only be modified through Actions, never directly
- Write a simple Action function that updates Store data

Content Outline:

- What an Action is: a named function with a clear, single responsibility
- Why direct mutation is dangerous: unpredictable side effects, hard-to-trace bugs
- The pattern: component calls an Action → Action updates the Store → Store change triggers re-render
- Naming conventions for Actions: use verbs (`addItem`, `removeUser`, `setFilter`)

Transitions: Students now understand the full MVC flow conceptually. The next lesson puts it into practice by building a mini Store together.

Key Principles:

- Actions are the only front door into the Store — everything else is read-only
- One Action, one responsibility: `addItem()` should not also validate, log, and redirect
- Name Actions like commands: `setUser`, `clearCart`, `toggleTheme`

Examples:

- `addToCart(product)` — an Action that takes a product and appends it to the cart array in the Store
 - `clearNotifications()` — an Action that sets the notifications array back to empty
 - Contrast: directly doing `store.cart.push(item)` versus calling `addToCart(item)` — the first is invisible, the second is traceable
-

01.3 Building a Mini Store 📖 (~600 words)

Learning Objectives:

- Implement a simple global Store using React Context and `useState`
- Write at least two Actions that modify the Store
- Connect a component to the Store and render data from it

Content Outline:

- Setting up a Context to act as the Store
- Defining the initial state
- Writing Actions as functions inside the Context Provider
- Consuming the Store in a child component using `useContext`
- Verifying that changes in the Store re-render the component

Transitions: With MVC understood and practiced, the next section shifts to a related but distinct question: what happens when data needs to survive a page refresh? That's where browser storage comes in.

Key Principles:

- Context is not magic — it's a container you fill with data and functions
- Keep the Store lean: only put in it what truly needs to be shared across multiple components

Hands-On Exercise: Build a simple notification center. The Store holds an array of notification messages. Students implement two Actions: `addNotification(message)` and `clearAll()`. A `<NotificationBell />` component reads the count from the Store and displays it. A `<NotificationList />` component renders each message. A button in another component triggers `addNotification()` to verify the Store updates are reflected everywhere.

Code Example:

```
// store/NotificationContext.jsx
const NotificationContext = createContext();

export function NotificationProvider({ children }) {
  const [notifications, setNotifications] = useState([]);

  const addNotification = (message) => {
    setNotifications((prev) => [...prev, message]);
  };

  const clearAll = () => setNotifications([]);

  return (
    <NotificationContext.Provider value={{ notifications, addNotification,
clearAll }}>
      {children}
    </NotificationContext.Provider>
  );
}
```

Success Criteria:

- The Store is defined in one place and not duplicated
- Actions are the only way to modify the Store's data
- Two separate components can read from the same Store simultaneously and stay in sync

Section 02: Browser Storage

02.0 LocalStorage vs SessionStorage 📖 (~500 words)

Learning Objectives:

- Define LocalStorage and SessionStorage and explain how each one works
- State the key difference: persistence across sessions vs. persistence within a session
- Identify the correct storage type for a given use case

Content Outline:

- What browser storage is: a key-value store built into every browser
- `LocalStorage`: persists until explicitly cleared — survives tab closes, browser restarts
- `SessionStorage`: persists only for the current tab session — wiped when the tab closes
- The API: `setItem`, `getItem`, `removeItem`, `clear` — identical for both
- Capacity limits: both store strings only, typically 5–10MB

Transitions: Now that students know *what* each storage type is, the next lesson focuses on the harder question: *when* should you use them at all?

Key Principles:

- `LocalStorage` outlives the session; `SessionStorage` dies with the tab
- Both store only strings — objects must be serialized with `JSON.stringify` and parsed with `JSON.parse`
- Neither is encrypted — treat both as visible to anyone with access to the browser

Examples:

- `LocalStorage`: user's preferred language, dark/light theme choice, a "don't show this again" dismissal flag
- `SessionStorage`: a multi-step form's progress within a single visit, a temporary filter selection that should reset on next visit

02.1 When to Store Something 📖 (~500 words)

Learning Objectives:

- Apply a decision framework to determine whether data should be stored in browser storage, the Store, or not at all
- Explain why not every piece of data deserves to be persisted
- Recognize the cost of over-storing: stale data, privacy risk, complexity

Content Outline:

- The core question: does this data need to survive a page refresh?
- If yes: does it need to survive closing and reopening the browser? → `LocalStorage` vs `SessionStorage`
- If no: keep it in component state or the Store
- The cost of storing too much: old data that contradicts new data from the server, privacy exposure, harder debugging
- A simple checklist: Is it user-specific? Is it non-sensitive? Does it need to persist? If all three: consider storage

Transitions: With the decision framework clear, the next lesson is hands-on — students will actually read from and write to browser storage in a React component.

Key Principles:

- Persistence is a feature that has a cost — only add it when the user genuinely benefits
- Never store what you can always re-fetch from the server
- Always have a plan for what happens when stored data is stale or missing

Examples:

- Worth storing in LocalStorage: a sidebar that the user collapsed — they expect it to stay collapsed next visit
 - Not worth storing: the current list of products — fetch it fresh from the API every time
 - Worth storing in SessionStorage: the current step in a checkout flow — if the tab closes, it's fine to start over
-

02.2 Reading and Writing to Storage 📖 (~600 words)

Learning Objectives:

- Write a React hook that reads from and writes to LocalStorage
- Correctly serialize and deserialize objects stored as strings
- Handle the case where stored data is missing or corrupted

Content Outline:

- Reading from storage: `localStorage.getItem(key)` and handling `null`
- Writing to storage: `localStorage.setItem(key, JSON.stringify(value))`
- Removing from storage: `localStorage.removeItem(key)`
- Building a custom hook `useLocalStorage(key, defaultValue)` that wraps this logic
- Why a custom hook: avoid repeating serialization logic everywhere

Transitions: Now that students can work with storage correctly, the next lesson teaches what *not* to do — the anti-pattern that makes storage dangerous.

Key Principles:

- Always provide a default value when reading — storage may be empty
- Always use `JSON.stringify` when writing objects and `JSON.parse` when reading them back
- Wrap storage logic in a hook to keep components clean

Hands-On Exercise: Build a `useLocalStorage` custom hook and use it in a `<ThemeToggle />` component. The component renders a button that switches between "light" and "dark" mode. The chosen theme is saved to LocalStorage so it persists across page refreshes. Students verify the persistence by switching the theme, refreshing the page, and confirming the choice was remembered.

Code Example:

```
function useLocalStorage(key, defaultValue) {
  const [value, setValue] = useState(() => {
    try {
      const stored = localStorage.getItem(key);
      return stored ? JSON.parse(stored) : defaultValue;
    } catch {
      return defaultValue;
    }
  });
}
```

```
const setStored = (newValue) => {
  setValue(newValue);
  localStorage.setItem(key, JSON.stringify(newValue));
};

return [value, setStored];
}
```

Success Criteria:

- The hook correctly returns the stored value on page load
- Updating the value through the hook updates both the component and LocalStorage
- If LocalStorage is empty, the default value is used without throwing an error

02.3 Anti-Pattern: Sensitive Data in Storage 📖 (~450 words)

Learning Objectives:

- Explain why browser storage is not safe for sensitive data
- List specific examples of data that should never be stored in LocalStorage or SessionStorage
- Apply the correct alternative for each type of sensitive data

Content Outline:

- Why browser storage is insecure: accessible via JavaScript, readable by any script running on the page (XSS risk)
- What counts as sensitive: passwords, authentication tokens, personal identification numbers, payment data
- The temptation: "it's so easy to just put the token in LocalStorage" — and why that's a trap
- What to do instead: HttpOnly cookies for auth tokens (handled server-side), never store passwords at all

Transitions: With browser storage understood correctly, the next section introduces how React's `use` hook lets you control *when* data is fetched and *what* triggers a re-fetch — the concept of data dependencies.

Key Principles:

- LocalStorage is readable by any JavaScript on the page — including injected malicious scripts
- A token in LocalStorage is a common and serious security vulnerability
- The rule: if you would be alarmed to see it in the browser console, don't store it in browser storage

Examples:

- ❌ Wrong: `localStorage.setItem('authToken', token)` — exposed to XSS attacks
- ❌ Wrong: `localStorage.setItem('password', userPassword)` — never store passwords anywhere client-side
- ✅ Right: store the user's preferred language, theme, or UI layout preferences — nothing that grants access or reveals identity

Section 03: Data Dependencies

03.0 What Is a Data Dependency? 📖 (~500 words)

Learning Objectives:

- Define what a data dependency means in the context of a React component
- Explain why uncontrolled dependencies cause bugs like infinite loops or stale data
- Identify the dependencies a component has by examining what values it reads

Content Outline:

- What a dependency is: any external value a piece of code relies on to produce its output
- Why dependencies matter in React: when a dependency changes, the component needs to know so it can update
- The problem of stale data: a component renders with old data because it didn't know a dependency had changed
- The problem of infinite loops: a component re-runs because it accidentally listed something as a dependency that it also changes

Transitions: With the concept of dependencies clear, the next lesson covers how to express and control them using the `use` hook.

Key Principles:

- A dependency is anything from outside the function that the function reads
- Unlisted dependencies lead to stale data; over-listed dependencies lead to infinite loops
- Thinking about dependencies is a skill — it gets easier with practice

Examples:

- A component that fetches user data based on a `userId` prop: `userId` is a dependency — if it changes, the fetch must re-run
 - A component that formats a date based on the user's locale preference: the locale is a dependency — if it changes, the formatted output must update
-

03.1 Controlling Dependencies with `use` 📖 (~500 words)

Learning Objectives:

- Explain how the `use` hook controls when data is fetched or recomputed
- Write a correct dependency array that reflects a component's actual dependencies
- Identify common mistakes: empty arrays when dependencies exist, missing dependencies

Content Outline:

- How `use` (and previously `useEffect`) uses a dependency array to decide when to re-run
- An empty dependency array `[]`: runs once on mount — use when there are truly no dependencies
- A populated array `[value1, value2]`: re-runs whenever any listed value changes
- No array at all: runs after every render — almost always a mistake

- How to identify what belongs in the dependency array: list every external value the hook reads

Transitions: The next lesson introduces one-way data flow — the principle that explains *how* data should move through a React tree, complementing what students now know about *when* it updates.

Key Principles:

- The dependency array is a contract: "re-run me when any of these change"
- React's linter (`eslint-plugin-react-hooks`) will warn about missing dependencies — listen to it
- If adding a dependency causes an infinite loop, the problem is usually mutation inside the hook, not the dependency list

Examples:

```
// Runs once on mount – no dependencies
use(() => { fetchGlobalConfig(); }, []);

// Re-runs whenever userId changes
use(() => { fetchUserProfile(userId); }, [userId]);

// Bug: runs on every render because no dependency array
use(() => { fetchData(); });
```

03.2 One-Way Data Flow 📖 (~500 words)

Learning Objectives:

- Define one-way data flow and explain why React enforces it
- Describe the correct direction: data flows down via props, events flow up via callbacks
- Identify anti-patterns that break one-way flow and explain their consequences

Content Outline:

- What one-way data flow means: data travels in one direction — from parent to child, never the reverse
- Data flows down: parents pass data to children through props
- Events flow up: children notify parents of changes through callback functions
- Why one-way flow makes apps predictable: you always know where a change came from
- What breaks this: prop drilling, children mutating parent data directly

Transitions: With all three concepts understood — MVC, browser storage, and data dependencies — the next lesson brings them together in a practical exercise before the assessment.

Key Principles:

- In React, data has one direction of travel: downward through props
- If a child needs to change something in a parent, it asks the parent to change it via a callback
- One-way flow makes debugging linear: follow the data up the tree to find where it changed

Examples:

- Data flowing down: `<ProductCard product={product} />` — the parent passes the product object to the child
 - Event flowing up: `<ProductCard onAddToCart={({id}) => dispatch(addToCart(id))} />` — the child calls the parent's callback when the user clicks
 - Anti-pattern: a child component directly modifying a shared array it received as a prop — now the parent doesn't know its data changed
-

03.3 Wiring Store, Storage, and Dependencies 📖 (~650 words)

Learning Objectives:

- Build a small app that uses a Store, reads from LocalStorage on mount, and respects data dependencies
- Correctly initialize the Store from persisted data in LocalStorage
- Use the `use` hook with the right dependency array to control when data is fetched or recomputed

Content Outline:

- The scenario: a preferences panel that remembers the user's choices
- Loading from LocalStorage on mount using `use` with an empty dependency array
- Saving to LocalStorage through an Action whenever a preference changes
- Using `use` with a dependency to react to changes in a specific Store value
- Verifying the full loop: change a preference → Store updates → LocalStorage saves → refresh → Store rehydrates from LocalStorage

Transitions: This exercise consolidates everything from Sections 01–03. Students are now ready for the assessment and the final review.

Key Principles:

- Initializing from storage on mount is a one-time operation — it belongs in a `use` with an empty dependency array
- Saving to storage should happen in an Action, not scattered across components
- The Store is the source of truth at runtime; LocalStorage is the backup for persistence

Hands-On Exercise: Build a `<UserPreferences />` panel with two settings: preferred language (English / Spanish) and display density (compact / comfortable). Requirements: (1) preferences are stored in a Context-based Store; (2) on app load, the Store is initialized from LocalStorage if saved values exist; (3) changing a preference calls an Action that updates both the Store and LocalStorage; (4) a `<PreviewPanel />` component reads from the Store and updates in real time when preferences change.

Success Criteria:

- Changing a preference updates the `<PreviewPanel />` immediately without a page refresh
 - Refreshing the page restores the last saved preferences
 - The LocalStorage write happens inside an Action, not directly inside the component
 - The `use` that loads from LocalStorage on mount has an empty dependency array `[]`
-

Section 04: Assessment

04.0 Data Flow Mini-App 📖 (~650 words)

Learning Objectives:

- Apply MVC architecture, browser storage, and data dependency concepts in a single cohesive mini-app
- Identify and fix at least one data flow bug introduced in a partially built app
- Demonstrate that the Store is the single source of truth and LocalStorage is used only for persistence

Content Outline:

- The scenario: a partially built reading list app with intentional data flow bugs
- Bug 1: a component reads data directly from LocalStorage instead of from the Store
- Bug 2: a child component mutates the items array directly instead of calling an Action
- Bug 3: a `use` hook has a missing dependency that causes stale data
- Students identify, explain, and fix each bug

Transitions: This exercise is the final practice before the assessment. It tests not just whether students can build, but whether they can read and reason about existing data flow.

Key Principles:

- Debugging data flow means following the data: where was it created, who changed it, who should have changed it
- A bug in data flow is almost always a violation of one of the three principles: single source of truth, controlled storage, correct dependencies

Hands-On Exercise: Students receive a reading list app with three pre-seeded bugs described above. For each bug they must: (1) identify what the bug is and why it's a problem, (2) explain what the correct approach is, and (3) fix the code. After all three fixes, the app should correctly: display items from the Store, persist the list to LocalStorage through Actions only, and re-fetch user preferences when the `userId` dependency changes.

Success Criteria:

- All three bugs are identified with a clear explanation of *why* each is a bug
 - Fixed code uses the Store as the single source of truth
 - LocalStorage reads and writes occur only through the designated hook or Action
 - The `use` dependency array correctly lists `userId` so the hook re-runs when it changes
-

04.1 Data Flow Assessment 🧐 (~700 words)

Learning Objectives:

- Demonstrate understanding of MVC architecture (Store and Actions)
- Demonstrate understanding of when and how to use LocalStorage vs SessionStorage
- Demonstrate understanding of data dependencies and one-way data flow

Question Format: Scenario-based (5 questions)

Questions:

Question 1 — Store vs Local State You're building a music player app. You have a `<VolumeSlider />` component and a `<NowPlaying />` component. The volume level needs to be accessible in both components and several others across the app. Where should the volume level live, and why?

Expected answer: In the global Store, because multiple components across the app need to read it. Local component state is only appropriate when a single component owns the data.

Question 2 — Choosing Storage Type A user completes step 2 of a 4-step checkout flow. You want to save their progress so they can continue if they accidentally navigate away within the same session — but if they close the browser and come back, they should start fresh. Which storage type do you use and why?

Expected answer: SessionStorage, because it persists within the current tab session but is cleared when the browser is closed. LocalStorage would incorrectly preserve the checkout state across sessions.

Question 3 — Actions and Direct Mutation A junior developer writes this code inside a component:

```
const { cart } = useStore();
cart.push(newItem); // add item directly
```

What is wrong with this approach? What should be done instead?

Expected answer: Direct mutation bypasses the Store's controlled update mechanism. React will not know the data changed, so the UI won't re-render. The correct approach is to call an Action: `addToCart(newItem)`, which updates the Store through `setState`, triggering a re-render.

Question 4 — Dependency Array A component fetches a user's order history based on a `userId` prop. Here is the hook:

```
use(() => {
  fetchOrders(userId);
}, []);
```

What is the bug? What happens as a result, and how do you fix it?

Expected answer: `userId` is missing from the dependency array. The hook runs once on mount with the initial `userId`. If `userId` changes later (e.g., an admin switches between user accounts), the hook does not re-run and the displayed orders are stale. Fix: change `[]` to `[userId]`.

Question 5 — One-Way Data Flow You have a `<FilterBar />` component that lets the user select a category. The selected category needs to be used by a `<ProductGrid />` component that is a sibling of `<FilterBar />`, not a child. How should data flow in this situation?

Expected answer: The selected category should be lifted to the shared parent component (or placed in the Store if the scope is app-wide). The parent passes the category down to `<ProductGrid />` as a prop.

`<FilterBar />` calls a callback prop (or dispatches an Action) when the user selects a category. Data flows down; events flow up.

Evaluation Criteria:

- Correctly identifies where data belongs (Store vs local state vs storage)
- Demonstrates understanding of why Actions are required for Store updates
- Correctly matches storage types to use cases
- Identifies missing dependencies and explains the resulting bug
- Describes one-way data flow correctly with lift-to-parent or Store as the solution

Score Interpretation:

- 5/5: Solid understanding of data flow — ready to build production-grade React apps
 - 4/5: Strong foundation with one gap — review the missed concept before moving on
 - 3/5: Core concepts understood but some confusion remains — revisit Sections 01–03
 - 2/5 or below: Recommend replaying the hands-on exercises before continuing
-

Section 05: What's Next

05.0 You Know How Data Moves (~450 words)

Content Outline:

What you accomplished: You started this course without a clear picture of where data should live in a React app. You now have a complete mental model. You understand that the Store is the single source of truth, Actions are the only way to modify it, and components are consumers — not owners — of shared data. You know when to reach for LocalStorage versus SessionStorage, what never to put in either, and how to persist data correctly through a custom hook. And you understand that React's data has a direction: it flows down through props and events bubble up through callbacks.

Why this matters: Every React app you'll ever build — from a simple tool to a full product — will need to answer the question "where does this data live?" You now have a framework for answering it consistently. That consistency is what separates apps that are easy to debug and extend from apps that become impossible to maintain.

Connection to the bigger picture: Data flow is the invisible architecture of your frontend. In the coming learnpacks, you'll start interacting with external APIs — fetching real data from a server, sending user actions back, and handling the full request-response cycle. Everything you've learned here about the Store, Actions, and dependencies will make that next step significantly smoother. When data arrives from an API, you'll know exactly where to put it and how to make it available to your components.

Next steps and continued learning:

- Explore Zustand or Redux Toolkit as production-ready Store implementations that follow the same MVC principles you learned here
- Practice the `use` hook's dependency array by deliberately introducing bugs and fixing them — it's the fastest way to build intuition

- Read the React documentation on Context and the `use` hook — now that you've used them in practice, the docs will make much more sense

Resources for further exploration:

- [React Docs — Context](#)
- [React Docs — `use` hook](#)
- [MDN — Web Storage API](#)
- [Zustand](#) — a minimal, modern alternative to Redux for global state

You've built a foundation that most developers only develop through painful trial and error. That's something to be genuinely proud of. On to the next one.